# NexHealth Integration

## 1. Overview

### 1.1 Description

**NexHealth Integration** connects **NexHealth** (dental and healthcare practice management platform) with **Newo Platform**.

It enables:

- Checking available appointment slots with intelligent provider, operatory, and appointment type matching
- Booking appointments with automatic patient record search and creation
- Cancelling existing appointments
- Multi-location support with AI-powered location selection
- Dynamic filtering by operatories and appointment types
- Auto-configuration of booking/cancellation scenarios on the agent canvas

This integration is designed for **dental clinics and healthcare practices** who need **automated appointment scheduling through a conversational AI agent** (voice or chat).

### 1.2 Use Cases

- When a patient asks about available slots → AI selects location, provider, operatory, and appointment type, then returns open time slots
- When a patient wants to book an appointment → Search or create patient record, match slot, and create appointment in NexHealth
- When a patient wants to cancel an appointment → Cancel the appointment via NexHealth API
- When a clinic has multiple locations → AI matches patient's natural language to the correct location
- When a new patient books → Automatically create patient record in NexHealth before booking
- When a conversation starts → Clinic locations are fetched and injected into the agent's context

### 1.3 Supported Features

| Feature | Supported | Notes |
|---|---|---|
| Check Availability | ✅ | Date/time/location with provider + operatory + type filtering |
| Book Appointment | ✅ | With automatic patient creation if needed |
| Cancel Appointment | ✅ | Requires booking_id from prior booking |
| Multi-Location Support | ✅ | AI-powered location matching from conversation |
| Provider Selection | ✅ | AI matches appointment preferences to providers |
| Operatory Filtering | ✅ | AI matches preferences to treatment rooms |
| Appointment Type Filtering | ✅ | AI matches preferences + new patient flag |
| Patient Search | ✅ | By phone number via NexHealth API |

| | | |
|---|---|---|
| Patient Creation | ✅ | Auto-creates during booking flow |
| Canvas Scenario Setup | ✅ | Auto-adds booking/cancellation scenarios |
| Token Auto-Refresh | ✅ | OAuth bearer token with automatic 401 retry |
| Reschedule Appointment | ❌ | Cancel + rebook as workaround |
| Historical Data Import | ❌ | Only new events after activation |

## 2. Requirements

Before installation:

- Active **NexHealth** account with API access
- **NexHealth API key** for OAuth authentication
- **NexHealth subdomain** (institution identifier)
- At least one location configured in NexHealth

## 3. How to Setup

### 3.1 Step 1 — Get API Credentials

1. Log in to your **NexHealth** account
2. Navigate to API settings or developer section
3. Generate or locate your **API key**
4. Note your **subdomain** (institution identifier in NexHealth)

### 3.2 Step 2 — Connect in Platform

1. Open **Newo → Projects**
2. Set the following attributes in **Builder / Attributes**:

| Attribute | Required | Description |
|---|---|---|
| `nexhealth_api_key` | ✅ | NexHealth API key for OAuth |
| `nexhealth_subdomain` | ✅ | Institution subdomain in NexHealth |
| `nexhealth_base_url` | ❌ | API base URL (default: `https://nexhealth.info`) |
| `nexhealth_enable_slot_check` | ❌ | Enable availability checks (default: `True`) |
| `nexhealth_enable_booking` | ❌ | Enable appointment creation (default: `True`) |
| `nexhealth_enable_cancellation` | ❌ | Enable cancellations (default: `True`) |
| `nexhealth_enable_filter_slots_by_operatories` | ❌ | Filter slots by operatories (default: `True`) |

| | | |
|---|---|---|
| `nexhealth_enable_filter_slots_by_appointments_type` | ❌ | Filter slots by appointment types (default: `True`) |
| `nexhealth_setup_scenarios` | ❌ | Auto-add booking/cancellation scenarios to canvas (default: `True`) |
| `nexhealth_override_agent_attributes` | ❌ | Override SuperAgent attribute schemas (default: `True`) |

3. Click **Save + Publish All**
4. On publish, the SetupFlow automatically:
    - Exchanges the API key for an OAuth bearer token
    - Creates HTTP connectors for API and token operations
    - Registers availability, booking, and cancellation tools with the agent
    - Injects payload schemas for the ConvoAgent tool calling framework

## 4. How to Use

This section explains how the integration works, how to configure automation, and how to test it.

### 4.1 How the Integration Works

The integration uses NexHealth's REST API (v2) for appointment management, patient records, and clinic data. Slot checking involves a multi-stage AI-powered selection pipeline: location → providers → operatories → appointment types → slots. The OAuth bearer token is auto-refreshed on 401 responses.

- A **Trigger** is a system event that starts a flow
- An **Action** is the API operation performed in NexHealth

**Available Triggers**

| Trigger | Event | Description |
|---|---|---|
| **Check Availability** | `get_available_slots` | When the agent needs to look up open slots |
| **Book Appointment** | `book_slot` | When the agent confirms a booking |
| **Cancel Appointment** | `convo_agent_cancel_booking` | When the agent processes a cancellation |
| **Session Start** | `session_started` | Fetches clinic locations for the session |
| **Setup** | `project_publish_finish` | Initializes integration on deploy |
| **Canvas Setup** | `gm_workflow_builder_canvas_built` | Auto-adds booking/cancellation scenarios |

**Available Actions**

- **Get Locations** — Fetch all clinic locations (GET `/locations`)
- **Get Providers** — Fetch doctors/providers at a location (GET `/providers`)
- **Get Operatories** — Fetch treatment rooms at a location (GET `/operatories`)
- **Get Appointment Types** — Fetch available service types (GET `/appointment_types`)
- **Get Slots** — Fetch available time slots (GET `/appointment_slots`)

- **Search Patient** — Find existing patient by phone (GET `/patients`)
- **Create Patient** — Create a new patient record (POST `/patients`)
- **Create Appointment** — Book an appointment slot (POST `/appointments`)
- **Cancel Appointment** — Cancel an existing appointment (PATCH `/appointments/{id}`)
- **Refresh Token** — Exchange API key for bearer token (POST `/authenticates`)

### 4.2 Architecture Diagrams

**Overall Sequence**

```
sequenceDiagram
    participant U as User
    participant A as ConvoAgent
    participant I as NexHealth Integration
    participant API as NexHealth API

    Note over I,API: Setup (on publish)
    A->>I: project_publish_finish
    I->>API: POST /authenticates (api_key)
    API-->>I: access_token
    I->>A: Tools registered

    Note over I,API: Session Start
    A->>I: session_started
    I->>API: GET /locations (subdomain)
    API-->>I: Locations list
    I->>A: Locations injected into context

    Note over U,API: Availability Check
    U->>A: "Any slots on Monday?"
    A->>I: get_available_slots (date, preferences)
    I->>I: AI select location
    I->>API: GET /providers (location_id)
    I->>I: AI select providers
    I->>API: GET /operatories (location_id)
    I->>I: AI select operatories
    I->>API: GET /appointment_types (location_id)
    I->>I: AI select appointment type
    I->>API: GET /appointment_slots (location, providers, operatories, type)
    API-->>I: Available slots
    I->>A: result_success (availability[])
    A->>U: "Here are the available times..."

    Note over U,API: Booking
    U->>A: "Book 10:00 AM please"
    A->>I: book_slot (customerInfo, dateTime)
    I->>API: GET /patients (phone)
    API-->>I: Patient results
    alt Patient not found
        I->>API: POST /patients (name, phone, DOB)
        API-->>I: New patient_id
    end
```

```
    I->>API: POST /appointments (patient, slot, location)
    API-->>I: appointment_id
    I->>A: result_success (booking_id)
    A->>U: "Your appointment is confirmed!"

    Note over U,API: Cancellation
    U->>A: "Cancel my appointment"
    A->>I: convo_agent_cancel_booking (booking_id)
    I->>API: PATCH /appointments/{id} (status: cancelled)
    API-->>I: Confirmation
    I->>A: result_success (cancelled)
    A->>U: "Your appointment has been cancelled"
```

**AvailabilityFlow (Multi-Stage Selection)**

```
flowchart TD
    E["get_available_slots"] --> LOC["AI select location<br>from
nexthealth_locations"]
    LOC --> PROV["GET /providers<br>{location_id, inactive=false}"]
    PROV --> PROV_AI["AI select providers<br>(active, bookable_online,<br>match
preferences)"]
    PROV_AI --> OPER{"Operatory<br>filter enabled?"}
    OPER -->|"Yes"| GET_OP["GET /operatories<br>{location_id}"]
    GET_OP --> OP_AI["AI select operatories<br>(active, bookable_online,<br>match
appt categories)"]
    OP_AI --> TYPE{"Appointment type<br>filter enabled?"}
    OPER -->|"No"| TYPE
    TYPE -->|"Yes"| GET_TYPE["GET /appointment_types<br>{location_id}"]
    GET_TYPE --> TYPE_AI["AI select type<br>(bookable_online,<br>new_patient flag)"]
    TYPE_AI --> SLOTS["GET /appointment_slots<br>{start_date, days=5,<br>lids[],
pids[],<br>operatory_ids[],<br>appointment_type_id}"]
    TYPE -->|"No"| SLOTS
    SLOTS --> PARSE["Parse slots:<br>convert to local time<br>group by date"]
    PARSE --> OUT["result_success<br>{availability[]}"]
```

**BookingFlow**

```
flowchart TD
    E["book_slot"] --> PHONE{"Phone number<br>available?"}
    PHONE -->|"No"| ASK["Send urgent_message:<br>request phone number"]
    PHONE -->|"Yes"| SEARCH["GET /patients<br>{phone_number,<br>location_id}"]
    SEARCH --> FOUND{"Patient<br>found?"}
    FOUND -->|"Yes"| USE["Use existing<br>customer_id"]
    FOUND -->|"No"| CREATE["POST /patients<br>{first_name, last_name,<br>phone, DOB,
email,<br>provider_id}"]
    CREATE --> NEW["Extract new<br>customer_id"]
    USE --> ENRICH["Match bookingDateTime<br>to stored slot data:<br>provider,
operatory,<br>start/end time"]
    NEW --> ENRICH
    ENRICH --> BOOK["POST /appointments<br>{patient_id, provider_id,<br>start_time,
```

```
end_time,<br>operatory_id}"]
    BOOK --> OK{"Success?"}
    OK -->|"No"| ERR["ResultError"]
    OK -->|"Yes"| OUT["result_success<br>{booking_id}"]
```

**CancellationFlow**

```
flowchart TD
    E["convo_agent_cancel_booking"] --> VAL{"booking_id<br>present?"}
    VAL -->|"No"| ERR["ResultError:<br>missing booking_id"]
    VAL -->|"Yes"| API["PATCH /appointments/{id}<br>{status: cancelled,<br>reason:
notes}"]
    API --> OK{"Success?"}
    OK -->|"No"| ERR2["ResultError"]
    OK -->|"Yes"| OUT["result_success<br>{cancel_booking}"]
```

**UtilsFlow (Token Refresh)**

```
flowchart TD
    REQ["Any API request"] --> HTTP["Send via<br>nexhealth_connector"]
    HTTP --> STATUS{"HTTP<br>status?"}
    STATUS -->|"200"| CHECK{"Response has<br>error field?"}
    CHECK -->|"No"| SUCCESS["nexhealth_result_success"]
    CHECK -->|"Yes"| ERR["nexhealth_result_error"]
    STATUS -->|"401"| REFRESH["POST /authenticates<br>{Authorization: api_key}"]
    REFRESH --> TOK_OK{"New token?"}
    TOK_OK -->|"Yes"| SAVE["Save token<br>Retry request<br>(max 3 attempts)"]
    TOK_OK -->|"No"| ERR
    STATUS -->|"4xx/5xx"| ERR
```

### 4.3 Where to Test

Testing can be done through the **Newo conversation interface** (voice or chat). Each flow (Availability, Booking, Cancellation, StaticData) has a dedicated webhook test endpoint for connectivity validation.

### 4.4 Testing and Verification

**To test the integration:**

1. **Setup**: Publish the project and verify token exchange succeeds (check `nexhealth_access_token` is populated)
2. **Locations**: Start a conversation — verify locations are loaded (agent should know about available clinics)
3. **Availability**: Ask "What slots are available on Monday?" — verify slots returned with correct times
4. **Booking**: Complete a booking conversation — verify appointment appears in NexHealth dashboard
5. **Cancellation**: Request cancellation of a booked appointment — verify status changed in NexHealth

**If no action occurs:**

- Ensure `nexhealth_api_key` and `nexhealth_subdomain` are set correctly
- Verify `nexhealth_access_token` was obtained (check it's not empty after setup)

- Confirm the relevant feature flags are enabled ( `nexhealth_enable_booking` , `nexhealth_enable_slot_check` , `nexhealth_enable_cancellation` )
- Check that locations were fetched on session start
- Re-publish the project if credentials were changed

> *Note: This integration performs real operations in NexHealth. Appointments created or cancelled through the agent are reflected in the live NexHealth system.*

## 5. FAQ

**Q: How does the agent select the right provider and operatory?** A: The integration uses a multi-stage AI pipeline. After location selection, it fetches all providers, operatories, and appointment types, then uses LLM (Gemini/GPT-4) with low temperature (0.2) to match each against the patient's stated appointment preferences and new patient status. Only active, online-bookable options are considered.

**Q: How many days of availability does the agent check?** A: By default, 5 days starting from the requested date. The API call uses `days=5` parameter.

**Q: What patient information is required for booking?** A: First name, last name, phone number (E.164 format), and date of birth are required. Email is optional. If the phone number is not available, the agent will ask the patient for it before proceeding.

**Q: What happens if the OAuth token expires?** A: The UtilsFlow automatically detects 401 responses and refreshes the token using the API key. The original request is retried up to 3 times. This is transparent to the conversation flow.

**Q: Does the integration support operatory and appointment type filtering?** A: Yes, both are enabled by default. You can disable them individually via `nexhealth_enable_filter_slots_by_operatories` and `nexhealth_enable_filter_slots_by_appointments_type` attributes.

**Q: What scenarios are auto-added to the canvas?** A: When `nexhealth_setup_scenarios=True` (default), the integration adds booking and cancellation scenarios with corresponding intents to the agent canvas on setup. This includes step-by-step conversation flows for scheduling and cancelling appointments.

**Q: Which LLM models are used?** A: Gemini 2.5 Flash for most operations (location selection, provider matching). GPT-4 for operatory filtering, appointment type matching, and scenario generation. Gemini 2.5 Pro for patient lookup processing.